HTTP NA LINHA DE COMANDO: CURL, WGET E NETCAT

*Um webhook parou. A aplicação jurava enviar; o parceiro jurava não receber. Dois dias de e-mail até alguém rodar `curl -v` e ver um **301** apontando para o endpoint novo — que a biblioteca HTTP, por padrão, **não segue em POST**. **O navegador esconde tudo; na linha de comando você vê o protocolo cru.***

082
GUIAS LINUX

TRÊS FERRAMENTAS, TRÊS TERRITÓRIOS — E A SAÍDA QUE RESOLVE O CASO

curl
DIAGNÓSTICO E API
Fala uma requisição e **mostra tudo**: qualquer método, cabeçalho e corpo. Interagir com API é aqui.

wget
DOWNLOAD E ESPELHO
Grava em arquivo por padrão, **retoma** o que caiu (`-c`) e insiste sozinho. ISO de 4 GB é aqui.

nc
CANO DE TCP/UDP
Não entende protocolo nenhum: move bytes entre o socket e o terminal. "Tem alguém nesta porta?" é aqui.

```
$ curl -v -X POST https://api.parceiro.com/webhook \
    -H 'Content-Type: application/json' -d '{"evento":"teste"}'
*  Connected to api.parceiro.com · TLS handshake ok
>  POST /webhook HTTP/2
>  content-type: application/json
<  HTTP/2 301
<  location: https://api.parceiro.com/v2/webhook
```

`*` nota do curl: DNS, TCP, TLS, reuso de conexão `>` linha **enviada** — a sua requisição `<` linha **recebida** — a resposta do servidor

CURL: AS OPÇÕES QUE APARECEM EM TODO COMANDO

kali@kali: ~

```
$ curl -s -H 'Authorization: Bearer TOKEN' \
    https://api.exemplo.com/v1/itens | jq -r '.[].nome'
$ curl -s -X POST https://api.exemplo.com/v1/itens \
    -H 'Content-Type: application/json' \
    --data @corpo.json
# -d implica POST e tipo FORMULÁRIO: sem o Content-Type
# explícito, a API recebe JSON rotulado errado e rejeita.
# -d remove quebras de linha; --data-binary @arq preserva.
$ curl -s -o /dev/null -w 'dns:%{time_namelookup} \
    tls:%{time_appconnect} ttfb:%{time_starttransfer}\n' URL
# o mtr do HTTP: ttfb alto com o resto baixo = o servidor
# está demorando para processar, e não a rede.
```

O BÁSICO

`-sS`	silencioso, **mas ainda mostra erro** — o par certo em script
`-L`	segue redirecionamento (o que faltava no webhook)
`-I · -o · -O`	só cabeçalhos · grava com o nome que você der · com o nome remoto
`-m · --retry`	timeout total · tenta de novo em falha transitória

A PEGADINHA QUE QUEBRA SCRIPT

`-f`	**Sem ele, o curl sai com 0 mesmo num 404 ou 500** — receber a resposta já é sucesso para ele. Sob `set -e`, um `500` passa como se tivesse dado certo. O `-f` faz o status HTTP virar status de saída: **obrigatório em automação**

NETCAT: O MESMO BINÁRIO ESCUTA E CONECTA

MODO CLIENTE — CONECTA A UMA PORTA

```
nc -zv exemplo.com 443 · nc -v smtp.exemplo.com 25
```

seu teclado e sua tela → stdin/stdout → nc cliente → TCP → serviço remoto :443

MODO SERVIDOR — ESCUTA NUMA PORTA

```
nc -lvnp 9000 > recebido.tar
```

quem conecta → bytes que chegam → nc -l à escuta → stdout → seu terminal ou um arquivo

Como o `nc` não formata nada, você fala o protocolo à mão: `printf 'GET / HTTP/1.1\r\nHost: exemplo.com\r\n\r\n' | nc exemplo.com 80`. E `-z` não envia dado: abre, confirma e fecha.

WGET: BAIXAR E ESPELHAR

`-c`	continua o download interrompido — a razão de usar `wget` para arquivo grande
`--mirror`	com `--convert-links --page-requisites --no-parent`: site offline sem subir na árvore por acidente
`--wait=1`	com `--random-wait`: espelhar sem parecer — nem funcionar como — negação de serviço

TLS SEM O CADEADO

`s_client`	`echo | openssl s_client -connect host:443 -servername host | openssl x509 -noout -subject -dates`
`-servername`	**obrigatório com SNI**: sem ele você inspeciona o certificado errado e diagnostica um problema que não existe

`nc` **escutando com** `-e /bin/bash` **entrega um shell a quem conectar** — é o *reverse shell* canônico, e por isso a `netcat-openbsd` do Kali **removeu o** `-e`. Use só em laboratório próprio e isolado; o `nc` não tem cifra nem autenticação. E `curl -k` **é marcador temporário, nunca configuração**: em produção, apaga o alerta de intermediário que você quer ouvir.

RESUMO RÁPIDO

- ✓ **O** `-v` **mostra requisição (** `>` **) e resposta (** `<` **) lado a lado** — é o que transforma "não funciona" em "responde 301 para cá".
- ✓ `curl` **inspeciona e automatiza API;** `wget` **baixa e espelha;** `nc` **é o cano de TCP.**
- ✓ `curl` **sai com 0 mesmo em 404 e 500:** `-f` é obrigatório em automação, e `-sS` mantém a saída limpa mostrando só erro.
- ✓ `-d` **implica POST e formato de formulário** — para JSON, o `Content-Type` explícito não é opcional.
- ✓ `-w` **decompõe a latência** em DNS, TCP, TLS e primeiro byte, sem instalar nada.
- ✓ **O mesmo** `nc` **escuta e conecta** — simetria que serve ao diagnóstico e ao shell reverso.

NO KALI

`curl` **e** `wget` **são o motor de metade das ferramentas ofensivas, e a mesma requisição muda de natureza pelo alvo:** buscar a release de uma ferramenta é trivial; automatizar milhares de requisições contra servidor alheio para mapear endpoints é ataque a aplicação web, com o enquadramento da Lei 12.737/2012 quando não há autorização. O `-A` (User-Agent) e os cookies (`-b` / `-c`) mantêm sessão num teste — e denunciam automação mal disfarçada num log. `nc` **é a ferramenta didática do shell reverso, e entendê-la é defesa:** o padrão a detectar é uma conexão de **saída**, iniciada pelo próprio servidor, para porta alta incomum — por isso firewall de saída importa tanto quanto o de entrada. ***Banner grabbing*** **é o reconhecimento mais barato e educado** (`nc -v host porta` colhe o que o serviço publica de propósito); vira problema ao escalar para faixas inteiras. Do lado defensivo, reduzir banners (`ServerTokens Prod`) diminui o que um levantamento colhe de graça. `socat` **é o** `nc` **com esteroides:** túnel cifrado e *relay* — pivô limpo num teste autorizado, e o mesmo canal por onde dados saem quando o firewall de saída deixa. **RHEL:** o `nc` costuma ser o `ncat` do Nmap, com o `-e` atrás de `--sh-exec`.